Problem 3

1. When is `__name__ == '__main__'` truthy, and when is it falsy?

Every module has a global variable called `__name__`. While a module is imported by another module, `__name__` is the module's own name (for a file `problem4.py`, that is the string `'problem4'`), so the expression `__name__ == '__main__'` is falsy. When a module is the file that was executed to start the program, Python gives it the special name `'__main__'` instead, so the expression is truthy while that module runs.

2. What problem are we trying to solve by putting code in that block?

Importing a module causes all of its top-level code to execute. The main function is to solve the issue of allowing some code to only run if explicitly excecuting while not run during import.

3. What is the scope and lifetime of a variable assigned in that block?

The `if` statement does not create a new scope. A variable assigned inside the `if` block is therefore an ordinary global variable in the module's global namespace, exactly as if it had been assigned at the top level of the file. It comes into existence when that assignment line runs and lives until the program ends, staying in the module's namespace the whole time. It is not local to the block and it is not destroyed when the block finishes.

Yes, this should affect the choice to assign variables there.

1. the variable is global, making them changable anywhere. which makes a program's behavior hard to trace and its functions hard to test.
2. the variable exists only if the block actually ran, so a function that relies on it breaks when the module is imported instead of executed, because then the assignment never happened and the name is undefined.